

메가존클라우드 Solution Architecture 양성 과정

컨테이너 DNS 응용 실습

컨테이너 DNS 응용

곽덕연

2025-2-27

[목차]

컨테이너 DNS 응용

<실습명>	2
실습 목표	2
시나리오1. 커맨드 정리	2
흐름00. 옵션 -d.....	2
흐름01. 옵션 -exec, -attach 차이	3
흐름02. 권한 연구(MYSQL_ROOT_PASSWORD / dh.sh - 권한) 오류! 책갈피가 정의되어 있지 않습니다.	
시나리오2. 자신의 웹서버 만들고, 이미지를 추가	3
흐름01. 사용할 VM 선택 및 도커 엔진 다운로드	3
흐름02. 웹서버 컨테이너 만들기	4
시나리오3. 도커 이미지 생성 및 백업	5
흐름01. 컨테이너 재확인	5
흐름02. 위에서 생성한 웹 서버 컨테이너를 도커 이미지로 저장(commit)	5
흐름03. 생성한 이미지를 도커 허브 (Public 영역)에 공유.....	5
흐름04. Private 영역 저장소를 위한 docker image 찾기 및 설치.....	7
흐름05. Private 영역 저장소 저장/태깅/push.....	7
시나리오4. 3개의 VM으로 DNS Server 구축.....	8
흐름01. 3개의 VM.....	8
흐름02. MainUS(물리적 DNS) 구현.....	9
흐름03. Extra VM 구현	9
흐름04. SubUS(논리적 DNS) 설치	10
흐름05. DNS 컨테이너 구현	11

<실습명>

Container를 활용한 백업 및 공유환경 구축

1. 실습 목표
 - Container의 개념 및 서비스를 통한 활용을 할 수 있다.
 - Container로 Web Server를 구축할 수 있다.
 - Container로 DNS Server를 구축할 수 있다.
2. 실습 환경 구축 및 시나리오 테스트1(도커 커맨드 정리)
 - 옵션 -d
 - 옵션 -exec, -attach 차이
 - 불필요 옵션을 정리
 - 권한 연구(MYSQL_ROOT_PASSWORD / dh.sh-실행 파일 권한)
3. 실습 환경 구축 및 시나리오 테스트2(자신의 웹서버 만들고, 이미지를 추가)
4. 실습 환경 구축 및 시나리오 테스트3(도커 이미지 생성 및 백업)
 - 앞서 생성한 웹 서버 컨테이너를 도커 이미지로 저장(commit)
 - 해당 이미지를 자신의 public docker hub에 업로드(백업)
 - 이미지 로컬 저장 - Public and Private Repository 사용
5. 실습 환경 구축 및 시나리오 테스트4(3개의 VM으로 DNS Server 구축)
 - VM1 : 도커 컨테이너 - 웹서버 / DNS 서버 : 논리적 분리
 - VM2 : DNS 서버(물리적 분리)
 - VM3 : 도커 컨테이너 - 웹서버 (조원의 웹 서버 이미지를 통해 구축)
 - DNS 요청 과정 정리
 - HTTP 요청 과정 정리

실습 목표

1. Container의 개념 및 서비스를 통한 활용을 할 수 있다.
2. Container로 Web Server를 구축할 수 있다
3. Container로 DNS Server를 구축할 수 있다.

시나리오1. 커맨드 정리

흐름00. 옵션 -d

- 어떤 도커 컨테이너 명령어들이 -d 옵션을 가지고 있을까요?
 - docker run , docker container start -d
 - 이외에도 docker compose up -d와 docker service create가 백그라운드 실행에 사용됩니다.
* Compose는 compose.yaml에 정의된 서비스를 실행할 때, Swarm의 docker service create는 서비스를 생성할 때 사용합니다.
- docker container run -d
 - 컨테이너를 백그라운드에서 실행

- docker container start -d
 - 중지된 컨테이너를 백그라운드에서 실행

흐름01. 옵션 -exec, -attach 차이

①. docker container exec

- 새로운 프로세스를 컨테이너 내부에서 실행
- 컨테이너 내부에서 새로운 셸이나 새로운 프로세스를 실행
- 기존 프로세스와는 독립적으로 실행됨
- 종료해도 컨테이너 자체에는 영향을 주지 않음

②. docker container attach

- 컨테이너의 표준 입출력(STDOUT, STDIN, STDERR)에 연결
- 기존 컨테이너의 주 프로세스(ENTRYPOINT 또는 CMD로 실행된 프로세스)에 연결
- Ctrl + C로 종료하면 컨테이너 자체도 중지될 수 있음

③. 정리

상황	추천 명령어
컨테이너 내부에서 새로운 명령을 실행하고 싶을 때	docker container exec
실행 중인 컨테이너의 프로세스와 직접 연결하고 싶을 때	docker container attach
컨테이너 내부에서 새로운 셸을 띄우고 싶을 때	docker container exec -it
로그를 실시간으로 보고 싶을 때	docker container attach

docker container exec → 새로운 프로세스 실행 (기존 프로세스와 독립적)

docker container attach → 기존 실행 중인 프로세스와 연결 (입출력 공유)

흐름02. 권한 참고(MYSQL_ROOT_PASSWORD / 실행 스크립트 권한)

①. SKIP

시나리오2. 자신의 웹서버 만들고, 이미지를 추가

흐름01. 사용할 VM 선택 및 도커 엔진 다운로드

①. 우분투 서버 OS 22.04 LTS 선택

- 패키지 다운로드용 NAT 연결 및 HO#1도 연결

②. 도커 엔진 다운로드

```
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
```

<pre> sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc sudo chmod a+r /etc/apt/keyrings/docker.asc </pre>
<pre> echo ₩ "deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu ₩ \$(. /etc/os-release && echo "\${UBUNTU_CODENAME:-\$VERSION_CODENAME}") stable" ₩ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null sudo apt-get update </pre>
<pre> sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin </pre>

흐름02. 웹서버 컨테이너 만들기

- ①. 사용할 컨테이너 이미지 선택하기
 - 과거 실습부터에서 사용한 아파치와 nginx가 있는 확인
 - docker search nginx , docker search apache2
 - 둘 모두 존재 이미지 확인.
 - 우분투와 같이 있는 apache 보단 nginx 사용. 심지어 official임
- ②. 도커 이미지 다운로드 및 컨테이너 생성
 - docker pull nginx
 - docker create --name duckweb -p 80:80 nginx:latest
 - docker start duckweb
 - 확인하기 docker ps
- ③. Nginx 컨테이너에 이미지 cp
 - 도커 nginx 컨테이너의 index.html 위치
/usr/share/nginx/html/index.html
 - docker cp 명령어로 VM 내부 파일을 컨테이너로 전송
- ④. Index.html 파일 수정
 - 컨테이너 내부로 접근
docker exec -it duckweb /bin/bash
 - 필요 시 파일 에디터와 ping 패키지를 설치합니다.
apt-get update && apt-get install -y vim iputils-ping
 - index.html 파일을 수정합니다.

<pre> docker cp [전송할 파일 위치] [컨테이너명]:[전송될 위치] </pre>
<pre> docker cp /home/duck/pork.jpg duckweb:/usr/share/nginx/html/ </pre>

<pre> <html> <head> <title>King-duckduck</title> </pre>

```
</head>
<body>
  
</body>
</html>
```

⑤. curl명령어로 VM에서 페이지 확인하기

```
curl localhost
```

컨테이너 생성 시 80:80 포트를 매핑했으므로 브라우저에서 포트 번호 없이 접속할 수 있습니다.

시나리오3. 도커 이미지 생성 및 백업

흐름01. 컨테이너 재확인

①. docker ps를 통하여 실행중인지 확인

- docker ps

흐름02. 위에서 생성한 웹 서버 컨테이너를 도커 이미지로 저장(commit)

①. 명령어 docker commit [OPTIONS] CONTAINER [REPOSITORY[:TAG]]

- CONTAINER는 커밋하려는 실행 중인 컨테이너의 ID 또는 이름.
- REPOSITORY는 새로 생성될 이미지 이름
- TAG는 새로 생성될 이미지의 태그.

②. commit으로 새로운 이미지 생성

```
docker commit duckweb duckweb:latest
docker images | grep duckweb
```

```
root@SubUS:~#
root@SubUS:~# docker commit -m "my first commit!!! docker create -p 8888:80 --name dweb nginx:latest" -a "duckduck" dweb duckweb:latest
docker commit -m "my first commit!!docker ps! docker create -p 8888:80 --name dweb nginx:latest" -a "duckduck" dweb duckweb:latest
sha256:70e540cbc75e3e84a0dd5027b4349de523c8afbc32230a40c5fe4f47ddcaed04
root@SubUS:~# docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
duckweb       latest   70e540cbc75e   5 seconds ago  255MB
sad           latest   a4001100a246   18 hours ago   284MB
nginx         latest   b52e0b094bc0   3 weeks ago    192MB
ubuntu/bind9  latest   5210e5bddb6a   5 months ago   168MB
root@SubUS:~#
```

③. 생성된 이미지 확인

```
docker images
```

흐름03. 생성한 이미지를 도커 허브 (Public 영역)에 공유

①. Docker Hub에 이미지를 올리려면 계정이 필요합니다.

②. docker hub에 가입하고 로그인

- 웹 브라우저에서 계정을 생성하고 로그인
- VM으로 진입하여 docker login -u [계정] 입력
- 비밀번호 입력하면 로그인 successful 표시

```
root@SubUS:~# docker login -u dajo075@gmail.com

Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings

Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@SubUS:~# docker push corntrinity/duckweb:latest
```

- ③. 이미지를 업로드하기 전에 Docker Hub 형식에 맞게 태그를 추가합니다.
 - Docker Hub는 <Docker Hub ID>/<이미지명>:<태그> 형식을 따름
 - docker tag duckweb:latest corntrinity/duckweb:latest

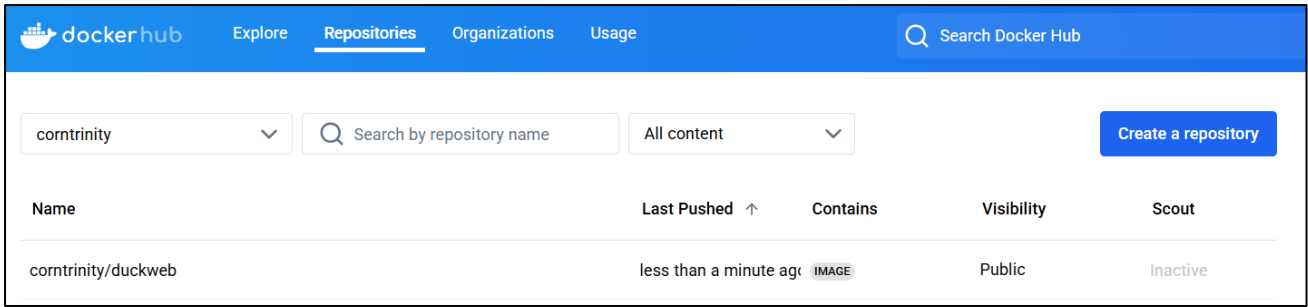
- ④. Docker Hub에 이미지 푸시

```
docker tag duckweb:latest corntrinity/duckweb:latest
docker push corntrinity/duckweb:latest
```

- ⑤. 확인

dockerhub Explore Repositories Organizations Usage				
Search Docker Hub				
corntrinity	Search by repository name	All content	Create a repository	
Name	Last Pushed ↑	Contains	Visibility	Scout
corntrinity/test_nginx	11 months ago	IMAGE	Public	Inactive
corntrinity/test_one	about 1 year ago ⚠		Public	Inactive
1-2 of 2 < >				

```
root@SubUS:~# docker push corntrinity/duckweb:latest
The push refers to repository [docker.io/corntrinity/duckweb]
59392fad2d7c: Pushing [=====] 48.17MB/62.85MB
55e9644f21c3: Mounted from library/nginx
7d22e2347c12: Mounted from library/nginx
f6d5815f290e: Mounted from library/nginx
791f0a07985c: Mounted from library/nginx
cabea05c000e: Waiting
c68632c455ae: Waiting
5f1ee22ffb5e: Waiting
```



흐름04. Private 영역 저장소를 위한 docker image 찾기 및 설치

①. 레지스트리 역할을 하는 이미지가 있는지 검색합니다.

- docker search registry로 레지스트리 이미지를 검색합니다.
- 공식 registry 이미지를 선택하고 Docker Hub 문서에서 역할과 사용법을 확인합니다.
- 맞춤 이놈을 다운로드
- docker pull registry

```
root@SubUS:~# docker pull registry
Using default tag: latest
latest: Pulling from library/registry
44cf07d57ee4: Pull complete
bbbdd6c6894b: Pull complete
8e82f80af0de: Pull complete
3493bf46cdec: Pull complete
6d464ea18732: Pull complete
Digest: sha256:a3d8aaa63ed8681a604f1dea0aa03f100d5895b6a58ace528858a7b332415373
Status: Downloaded newer image for registry:latest
docker.io/library/registry:latest
root@SubUS:~#
```

- 저장소용 컨테이너 실행

```
docker run -d -p 50000:5000 --restart always --name registry registry:2

curl -X GET http://localhost:50000/v2/_catalog
```

흐름05. Private 영역 저장소 저장/태깅/push

①. docker images로 기존에 commit한 이미지 존재 확인

- 기존 이미지가 없으면 실행 중인 컨테이너에서 다시 commit하거나 Docker Hub에서 다시 pull합니다.
- 컨테이너가 계속 구동 중이면 다시 commit하여 이미지를 생성할 수 있습니다.

```
root@SubUS:~# docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
corntrinity/duckweb latest       70e540cbc75e     9 hours ago     255MB
```


②. 도커 이미지를 로컬 레지스트리용으로 태깅합니다.

- `docker tag [이미지명] localhost:[연결된포트]/[이미지명]`

```
docker tag duckweb:latest localhost:50000/duckweb:latest
```

```
root@SubUS:~# docker image ls
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
localhost:50000/duckweb latest       70e540cbc75e   22 minutes ago 255MB
conrtrinity/duckweb latest       70e540cbc75e   22 minutes ago 255MB
```

③. 도커 이미지를 로컬 레지스트리에 push합니다.

- `docker push localhost:[연결된포트]/[이미지명]`

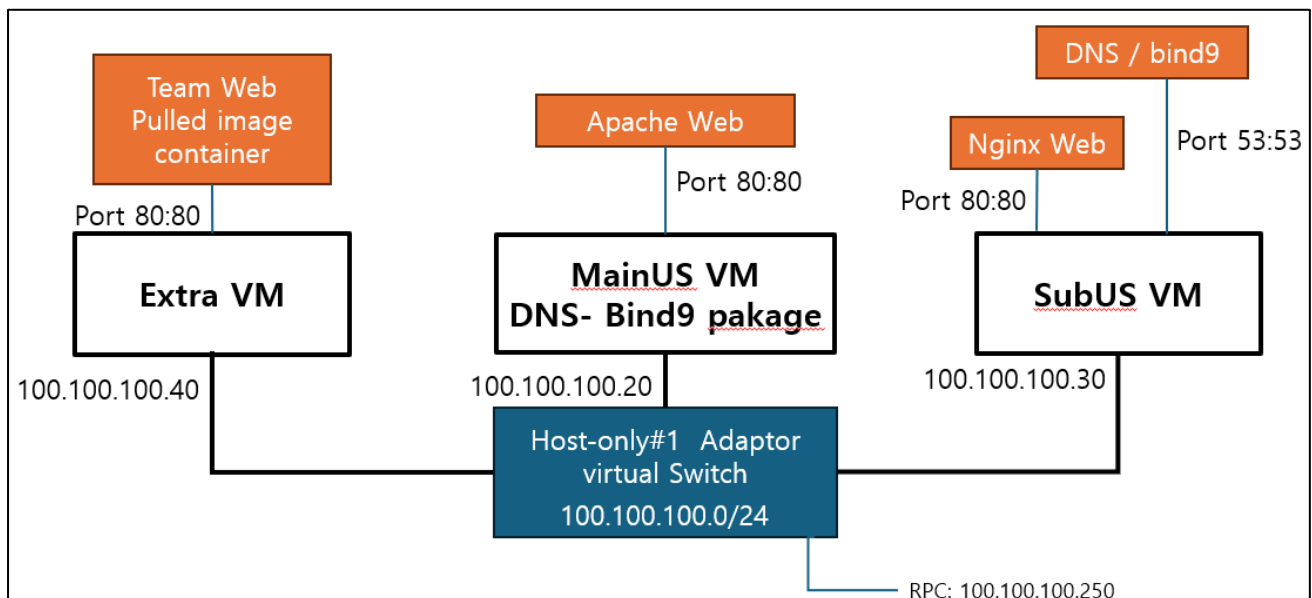
```
docker push localhost:50000/duckweb:latest
curl -X GET http://localhost:50000/v2/_catalog
```

④. 로컬 레지스트리에 이미지가 저장되었는지 catalog API로 확인합니다.

- `curl -X GET http://localhost:50000/v2/_catalog`

시나리오4. 3개의 VM으로 DNS Server 구축

구성도



흐름01. 3개의 VM

1. MainUS VM:

물리적 DNS 서버. 퍼블릭에 위치 포워딩을 통해 사설 DNS는 숨기는 역할

2. SubUS VM:

논리(컨테이너) DNS 서버. 사설망에 위치. 두 VM의 웹 컨테이너 가짐.

3. Extra VM:

조원들이 퍼블릭으로 올린 웹 컨테이너를 이용.

기본적으로 모든 VM은 MainUS를 DNS 서버로 지정하지만, 실제 질의 처리는 SubUS의 컨테이너 DNS로 전달됩니다.

흐름02. MainUS(물리적 DNS) 구현

①. `sudo apt update && sudo apt install -y bind9 bind9utils bind9-doc dnsmasq`

②. 설정 파일 수정. SubUS로 넘겨주기(Forward)

Forwarders 설정: `sudo vim /etc/bind/named.conf.options`

```
options {
    directory "/var/cache/bind";
    recursion yes;
    allow-query { localhost; 100.100.100.0/24; };
    allow-recursion { localhost; 100.100.100.0/24; };
    forwarders { 100.100.100.30; };
    dnssec-validation no;
    listen-on { any; };
    listen-on-v6 { none; };
};
```

③. 도커 엔진 설치

```
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo ₩
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu ₩
$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | ₩
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

④. 아파치 컨테이너 및 이미지 삽입: SKIP

흐름03. Extra VM 구현

①. 팀원이 올린 이미지 검색 및 다운

NAME	DESCRIPTION	STARS	OFFICIAL
moon4ang/web-hyeok		0	
dashyeok/dashyeok	Dashyeok Dashyeok	0	

```

root@US-docker:/home/duck#
root@US-docker:/home/duck# docker pull moon4ang/web-hyeok
Using default tag: latest
latest: Pulling from moon4ang/web-hyeok
7cf63256a31a: Already exists
bf9acace214a: Already exists
513c3649bb14: Already exists
d014f92d532d: Already exists
9dd21ad5a4a6: Already exists
943ea0f0c2e4: Already exists
103f50cb3e9f: Already exists
c751a9f16017: Pull complete
Digest: sha256:d963f3ace856894c5d09499aaecabb790562877fe8388db0332b650bb7e6b866
Status: Downloaded newer image for moon4ang/web-hyeok:latest
docker.io/moon4ang/web-hyeok:latest
root@US-docker:/home/duck#

```

②. 시작

```

root@US-docker:/home/duck# docker create -p 8888:80 --name 4angweb moon4ang/web-hyeok
429fa69114c6a9eac23609eed029819d77bab17147b16cc5329ccf8649c2f086
root@US-docker:/home/duck#

```

③. 확인

```

root@US-docker:/home/duck# docker create -p 8888:80 --name 4angweb moon4ang/web-hyeok
429fa69114c6a9eac23609eed029819d77bab17147b16cc5329ccf8649c2f086
root@US-docker:/home/duck#
root@US-docker:/home/duck#
root@US-docker:/home/duck#
root@US-docker:/home/duck# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
429fa69114c6   moon4ang/web-hyeok  "/docker-entrypoint..."  21 seconds ago  Created              4angweb
57cb408f138e   ubuntu/bind9:latest  "docker-entrypoint.sh"    25 hours ago   Exited (0) 24 hours ago  ddns
b8b376596f91   nginx:latest      "/docker-entrypoint..."  28 hours ago   Exited (0) 24 hours ago   web
root@US-docker:/home/duck# docker start 4angweb
4angweb
root@US-docker:/home/duck# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
429fa69114c6   moon4ang/web-hyeok  "/docker-entrypoint..."  36 seconds ago  Up 3 seconds    0.0.0.0:8888->80/tcp, [::]:8888->80/tcp  4angweb
root@US-docker:/home/duck# curl localhost:8888
<!DOCTYPE html>
<html>
<head>
<title>Welcome to Nginx!</title>
</head>
<body>
<h1>Welcome to SANGHYEOK HOMEPAGE!!!!</h1>

</body>
</html>
root@US-docker:/home/duck#

```

흐름04. SubUS(논리적 DNS) 설치

- ①. 웹 컨테이너 설정: SKIP
- ②. 도커 엔진 및 Nginx 컨테이너 설치/실행/이미지 삽입: SKIP

③. DNS 컨테이너 찾기

```
docker search bind9
```

→ ubuntu/bind9 패키지 설치

```
docker pull ubuntu/bind9
```

```
docker create --name dock_dns -p 53:53/tcp -p 53:53/udp ubuntu/bind9:latest
```

④. 포트 53 충돌 방지를 위해 VM의 systemd-resolved를 비활성화합니다.

```
sudo systemctl disable --now systemd-resolved
```

```
sudo rm /etc/resolv.conf
```

```
sudo bash -c 'echo "nameserver 127.0.0.1" > /etc/resolv.conf'
```

⑤. bind9 컨테이너 시작

```
docker start dock_dns
```

흐름05. DNS 컨테이너 구현

①. BIND9 기본 설정 파일 수정

IPv4만 사용하도록 설정합니다.

```
vim /etc/default/named
OPTIONS="-u bind -4"

# startup options for the server
OPTIONS="-u bind -4"
```

②. 기본 옵션설정

```
vim /etc/bind/named.conf.options
```

```
options {
    directory "/var/cache/bind";
    dnssec-validation auto;
    recursion yes;
    allow-query { any; };
    allow-recursion { any; };
    listen-on { any; };
    allow-transfer { none; };
    listen-on-v6 { none; };
};
```

```
options {
    directory "/var/cache/bind";

    dnssec-validation auto;

    recursion yes;
    allow-recursion { any; };
    listen-on { 172.17.0.3; };
    allow-transfer { none; };
    listen-on { any; };
    listen-on-v6 { none; };
};
```

③. Zone 명시

```
zone "team3.com" {
    type master;
    file "/etc/bind/zones/team3.com.db";
};

//include "/etc/bind/zones.rfc1918";

zone "team3.com"{
    type master;
    file "/etc/bind/zones/team3.com.db";
};
```

④. Zone 파일 영역 생성

Zone 파일 폴더 생성: `mkdir -p /etc/bind/zones/`

재사용할 파일 복사하여 Zone 파일 생성

`cp /etc/bind/db.local /etc/bind/zones/team3.com.db`

Zone 파일 설정: `vim /etc/bind/zones/team3.com.db`

```
$TTL 604800
@ IN SOA ns1.team3.com. root.team3.com. (
    2025022701 ; Serial
    604800     ; Refresh
    86400      ; Retry
    2419200    ; Expire
    604800 )   ; Negative Cache TTL
;
@ IN NS ns1.team3.com.
ns1 IN A 100.100.100.30
@ IN A 100.100.100.30
apache IN A 100.100.100.20
ex IN A 100.100.100.40
www IN CNAME apache.team3.com.
```

```

;
$TTL      604800
@         IN      SOA      team3.com. root.team3.com. (
                        2      ; Serial
                        604800 ; Refresh
                        86400  ; Retry
                        2419200 ; Expire
                        604800 ) ; Negative Cache TTL
;Basic Local Nginx
@         IN      NS       100.100.100.30
@         IN      A        100.100.100.30
;apache2 Web Conn
apache    IN      A        100.100.100.20
;docker-other-web
ex        IN      A        100.100.100.40

```

구문 오류 체크: named-checkconf와 named-checkzone 실행

```

root@c57293fb73ea:/etc/bind/zones# named-checkzone team3.com /etc/bind/zones/team3.com.db.
zone team3.com/IN: loaded serial 2
OK

```

⑤. 변경 설정 적용 및 정상 작동 확인

named-checkconf && named-checkzone team3.com /etc/bind/zones/team3.com.db

컨테이너 종료됨...

docker restart dock_dns

```

root@SubUS:~# docker start docker_dns
docker_dns
root@SubUS:~# docker exec -it docker_dns /bin/bash
root@c57293fb73ea:/#
root@c57293fb73ea:/#

```

⑥. 컨테이너의 nameserver 확인 및 수정

vim /etc/resolv.conf

```

# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 10.128.128.128
search

# Based on host file: '/run/systemd/resolve/resolv.conf' (legacy)
# Overrides: []
~

```

자기 자신을 가리키도록 설정합니다.

nameserver 127.0.0.1

흐름06. Docker/DNS 검증 체크리스트

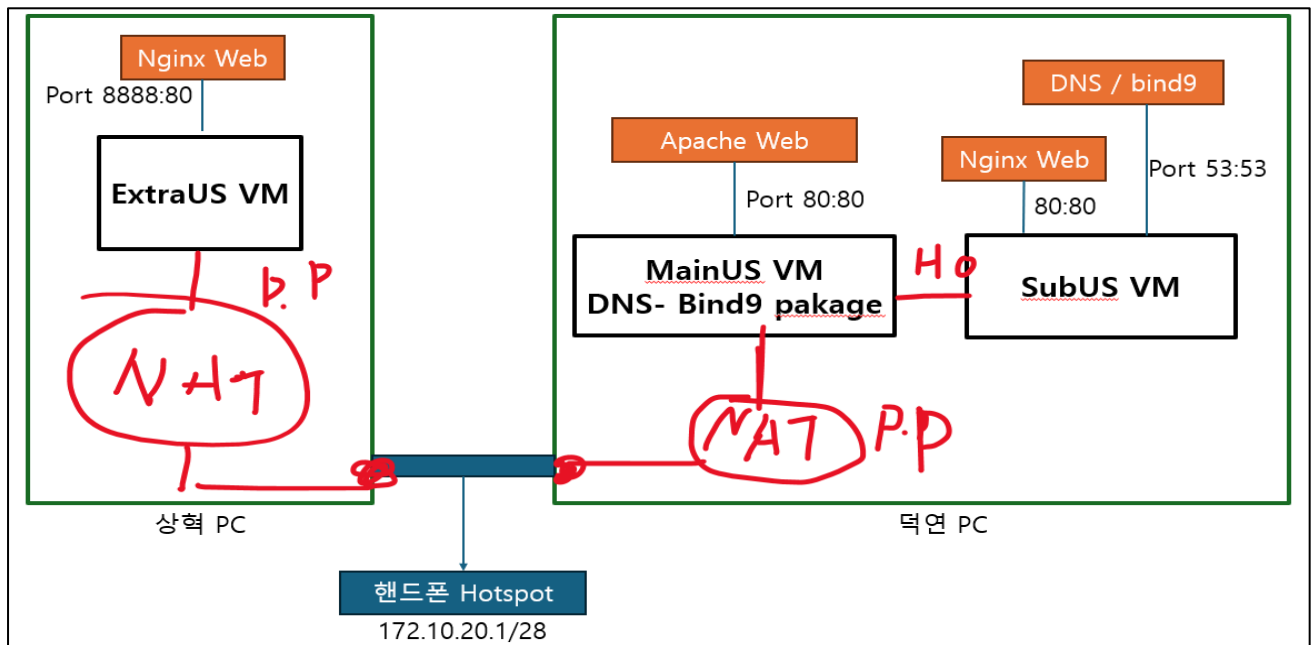
구분	검증 명령/방법	정상 기준
Docker 엔진	docker version / docker info	Client/Server 정보가 정상 출력
Web 컨테이너	docker ps --filter name=duckweb	duckweb 컨테이너 Up 상태
Web 접속	curl http://localhost	수정한 HTML 또는 이미지 응답 확인
이미지 생성	docker images grep duckweb	duckweb:latest 이미지 확인
Public push	docker push <DockerHubID>/duckweb:latest	push complete 확인
Private registry	curl http://localhost:50000/v2/_catalog	duckweb repository 표시
DNS 컨테이너	docker ps --filter name=dock_dns	dock_dns 컨테이너 Up 상태
BIND 문법	named-checkconf	오류 없이 종료
Zone 검증	named-checkzone team3.com /etc/bind/zones/team3.com.db	OK 출력
DNS 조회	dig @127.0.0.1 www.team3.com	100.100.100.20 또는 설정한 레코드 응답

```
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 127.0.0.1
search

# Based on host file: '/run/systemd/resolve/resolv.conf' (legacy)
# Overrides: []
```

HYPER EXTRA MISSION. 첫 짝꿍 수업



추가 실험 메모: 공인/사설 영역 간 포트포워딩과 DNS 흐름을 검증했습니다.

각 PC가 자신의 VM 서비스에 접근할 수 있도록 포트포워딩 규칙을 설계해야 합니다.

→ Hotspot으로 동적 설정된 IP를 확인하고 이를 Virtual Box Nat-network 설정에서 P.P함

NAT Network를 사용한 이유: DNS 서버/포워딩 실험 중 일반 NAT의 포트포워딩과 IP 할당 조건이 제한적이었기 때문입니다.

MainUS는 외부 접근을 위한 NAT와 내부 전달을 위한 Host-Only 어댑터를 함께 사용합니다.

Extra VM은 외부 영역이므로 NAT를 사용하고, SubUS는 사설 DNS 역할이므로 Host-Only로 내부망에만 연결합니다.

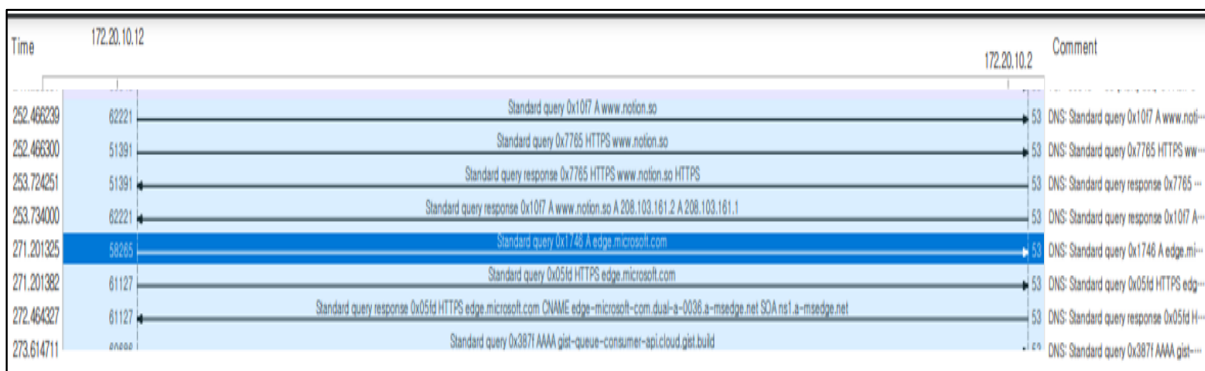
team3.com.db 존파일도 수정. 부여받은 공인IP에 맞게 수정

TCP 3-way handshake와 DNS flow를 확인했습니다.

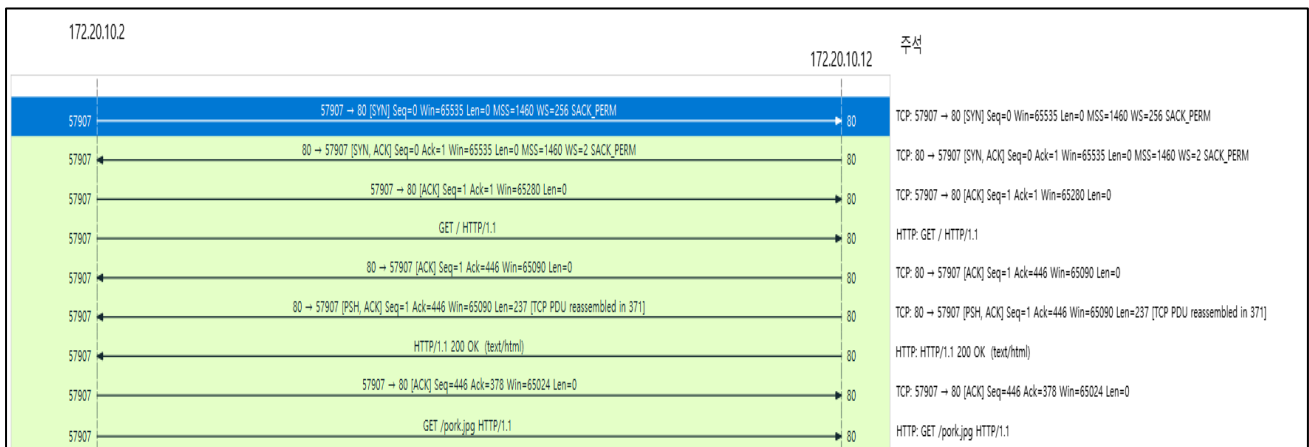
DNS 질의: Extra VM → Main US → Sub VM → Main US → Extra VM

TCP 핸드셰이크: 내부 접근이라 Wireshark로 못봄 tcpdump.

웹 접속 PC에서 DNS 요청 및 HTTP 접속 성공을 확인했습니다.



DNS 제공 PC에서 HTTP 3-way handshake와 GET 요청 흐름을 확인했습니다.



문제 정리(TS)

문제	확인	조치
컨테이너 이름 불일치	<code>docker ps -a grep dns</code>	dock_dns로 생성/시작/재시작 명령어 통일
53번 포트 충돌	<code>sudo ss -lntup grep :53</code>	systemd-resolved 비활성화 또는 포트 매핑 변경
Zone 파일 경로 오류	<code>grep team3.com /etc/bind/named.conf*</code>	team3.com.db 뒤의 불필요한 점(.) 제거
BIND 서비스 재시작 실패	<code>docker logs dock_dns</code>	named-checkconf, named-checkzone 통과 후 컨테이너 재시작
DNS 질의 실패	<code>dig @127.0.0.1 team3.com</code>	listen-on, allow-query, nameserver 설정 확인
이미지 push 실패	<code>docker login, docker tag, docker push</code> 순서 확인	태그 형식을 DockerHubID/이미지명:태그 또는 localhost:포트/이미지명:태그로 통일